

DOSSIER TECHNIQUE

Déployer et sécuriser les applications informatiques

Bloc 3

Application CESIZen

Application de gestion du bien-être mental

Clément Souil

CESI — Titre CDA (Concepteur Développeur d'Applications)

Juillet 2026

Table des matières

Table des matières

Table des matières	2
1. Présentation du projet CESIZen	3
1.1 Contexte	3
1.2 Contraintes du projet	3
1.3 Architecture technique	3
1.4 Fonctionnalités implémentées	4
1.5 Qualité logicielle — tests automatisés	4
2. Plan de déploiement	4
2.1 Stratégie de versioning — Git Flow adapté	4
2.2 Protection des branches	5
2.3 Environnements de déploiement	5
2.4 Pipeline CI/CD — GitHub Actions	6
2.5 Verrou de déploiement — GitHub Environments	6
2.6 Validation du pipeline par la démonstration	7
2.7 Déploiement Railway — rollback et continuité	7
3. Plan de maintenance	7
3.1 Outil de gestion des tickets — GitHub Issues	7
3.2 Méthodologie de gestion — flux prestataire ↔ client	8
3.3 Monitoring automatisé — SIEM	8
3.4 Versioning sémantique	9
4. Plan de sécurisation	9
4.1 Modélisation des menaces — STRIDE	9
4.2 Analyse des vulnérabilités — OWASP Top 10	9
4.3 Défense en profondeur — chaîne d'analyse	10
4.4 Actions correctives implémentées	10
4.5 Actions correctives planifiées (v2.0.0)	10
4.6 Authentification et contrôle d'accès — fonctionnement	10
4.7 Protection des données personnelles — RGPD	11
4.8 Gestion de crise	11
4.9 Sécurité de la chaîne d'approvisionnement (supply chain)	12
4.10 Bonnes pratiques de développement	12
5. Veille technologique	13
5.1 Méthode et organisation	13
5.2 Sources principales	13
5.3 Actions concrètes issues de la veille	13
6. Conclusion et couverture des critères	14
Annexe A — Documents complémentaires	15
Annexe B — Glossaire	16

1. Présentation du projet CESIZen

1.1 Contexte

CESIZen est une application de bien-être mental développée dans le cadre du titre Concepteur Développeur d'Applications (CDA) au CESI. Elle répond à un besoin croissant de solutions numériques accessibles pour la gestion du stress et la santé mentale.

L'application propose trois modules fonctionnels : gestion de comptes utilisateurs avec contrôle d'accès basé sur les rôles (RBAC), espace de ressources documentaires catégorisées, et diagnostic de stress basé sur l'échelle scientifique Holmes & Rahe (43 événements pondérés). Le Bloc 2 (conception et développement) a été soutenu le 19/05/2026 ; le présent dossier couvre le Bloc 3 : déploiement, maintenance et sécurisation de cette application.

1.2 Contraintes du projet

Les choix présentés dans ce dossier découlent directement des contraintes du contexte :

Contrainte	Conséquence sur les choix
Projet individuel (1 développeur)	Outils simples à opérer seul, automatisation maximale, pas d'astreinte 24/7 possible
Budget étudiant quasi nul	Tiers gratuits ou à faible coût — exclusion de Kubernetes managé et des suites payantes
Calendrier ~3 mois	Mise en œuvre rapide privilégiée sur l'architecture idéale ; limites documentées
Démonstration live exigée	L'environnement de production doit être réellement déployé, stable et accessible
Blocage constaté sur Azure for Students	MySQL indisponible dans les régions EU testées → choix du PaaS Railway

1.3 Architecture technique

Composant	Technologie	Port	Déploiement
API REST	.NET 9 ASP.NET Core	5289	Railway (service API)
Frontend Web	React 18 + Vite + TypeScript + Tailwind	5173	Railway (service web)
App Mobile	React Native + Expo	—	Expo Go (démon école)
Base de données	MySQL 8	3306	Railway (réseau interne uniquement)
ORM	Entity Framework Core (Code First)	—	Migrations automatiques au démarrage
Authentification	JWT Bearer (HMAC-SHA512, clé 512 bits min)	—	GitHub Secrets / Railway Variables

En production, seuls les services web et API exposent un port public (HTTPS 443, TLS géré par Railway via Let's Encrypt). MySQL n'est accessible que depuis le réseau interne Railway — aucune connexion directe depuis Internet n'est possible, ce qui réduit la surface d'attaque.

1.4 Fonctionnalités implémentées

- Module 1 — Comptes utilisateurs : inscription, connexion, déconnexion, changement de mot de passe, gestion admin (CRUD, rôles, activation/désactivation)
- Module 2 — Ressources documentaires : affichage avec filtres et recherche, création/modification/suppression/approbation (admin)
- Module au choix — Diagnostics Holmes & Rahe : 43 événements chargés depuis l'API, calcul de score, interprétation (Faible/Modéré/Élevé), historique utilisateur, configuration admin
- 3 plateformes — Web + Mobile + API déployée en production Railway

1.5 Qualité logicielle — tests automatisés

Le projet applique une approche TDD (Red → Green → Refactor). La couverture de tests s'établit à 47 tests unitaires automatisés :

Suite	Nombre	Périmètre
xUnit (.NET, backend)	39	Modèles User (14), StressResults (14), StressEvent (11) — cas nominaux, valeurs limites, cas d'échec
Vitest (React, frontend)	8	Logique réelle d'interprétation du score (getInterpretation) — bornes exactes des seuils 149/150 et 299/300

Les 39 tests backend s'exécutent à chaque CI (job API — Build & Test). Les 8 tests frontend s'exécutent localement via npm test ; leur intégration au pipeline est une évolution identifiée (section 6).

L'efficacité de cette couverture n'est pas seulement déclarée : la branche de démonstration demo/pipeline-fail-tests (PR #16) introduit volontairement une incohérence de seuil (100 au lieu de 150) et prouve que le pipeline bloque immédiatement le merge — le test de borne ScoreExactly150_ShouldBeModerateStress échoue et la protection de branche rend la fusion impossible.

2. Plan de déploiement

2.1 Stratégie de versioning — Git Flow adapté

Le projet utilise Git avec un schéma de branches inspiré de Git Flow, hébergé sur GitHub (Clement-Souil/CesiZen). La branche master représente le code en production, develop sert de branche d'intégration. Toutes les évolutions sont développées sur des branches dédiées.

Type	Nom de branche	Usage
Protégée	master	Production — PR obligatoire + CI verte
Protégée	develop	Intégration — PR obligatoire + CI verte
Travail	feature/*	Nouvelles fonctionnalités
Travail	fix/*	Corrections de bugs
Travail	security/*	Correctifs sécurité
Travail	test/*, docs/*	Renforcement de tests, documentation

Ce schéma structure l'historique réel du projet — chaque évolution significative a transité par une Pull Request dédiée :

PR	Branche source	Contenu
#22	security/trivy-scan-frontend	Extension du scan Trivy aux dépendances npm du frontend
#20	test/stress-score-boundaries	Tests de bornes Vitest sur les seuils Holmes & Rahe
#19	master → develop	Resynchronisation de develop (22 commits d'écart rattrapés)
#18	feature/deployment-lock	Gate de déploiement + monitoring SIEM
#15–#17	demo/pipeline-*	3 scénarios de démonstration du pipeline (voir 2.6)
#14	feature/deploy-frontend	Déploiement Railway du frontend React
#13	security/jwt-trivy-sonar	Sortie de la clé JWT du code + intégration Trivy et SonarCloud
#12	fix/auto-migrate-on-startup	Application automatique des migrations EF Core au démarrage

2.2 Protection des branches

Les règles suivantes sont actives sur master et develop, vérifiables en direct sur GitHub :

- Aucun commit direct — enforce_admins actif : la règle s'applique aussi au propriétaire du dépôt
- Pull Request obligatoire avant tout merge ; en contexte individuel, aucune approbation tierce n'est exigée (GitHub interdit l'auto-approbation de sa propre PR) — la contrainte bloquante est la CI verte
- Quatre status checks requis : API — Build & Test, Web — Lint & Build, Security — Trivy Scan, Quality — SonarCloud Analysis
- Force push et suppression de branche interdits ; branche à jour exigée avant merge (strict)

Convention de commit : feat:, fix:, ci:, security:, test:, docs: — l'historique reste lisible et exploitable pour générer des changelogs.

2.3 Environnements de déploiement

Critère	Développement	Test / CI	Production
Objectif	Dev local	Validation automatique	Mise en production
API	localhost:5289	GitHub Actions (build)	Railway HTTPS
Web	localhost:5173	GitHub Actions (build)	Railway HTTPS
Base de données	MySQL WampServer	Non connectée (tests isolés)	MySQL Railway
Secrets	appsettings.Development.json	GitHub Secrets	Variables Railway
Déclenchement	Manuel	Push / Pull Request	Merge master (auto)
Durée de vie	Permanent (poste dev)	Éphémère (durée du run)	Permanent (24/7)

La séparation des secrets par couche est stricte : aucun secret dans le code versionné (la clé JWT en a été retirée — PR #13), GitHub Secrets pour la CI, Variables Railway pour la production. La rotation d'un secret ne nécessite aucun redéploiement de code.

Secret	Développement	CI	Production
Clé JWT	appsettings.Development.json (non versionné)	GitHub Secret JWT_KEY	Variable Railway JWT_KEY
Connexion MySQL	appsettings.Development.json	Non requise (tests isolés)	Variable Railway DATABASE_URL
SONAR_TOKEN	—	GitHub Secret	—
GITHUB_TOKEN	—	Généré automatiquement	—

2.4 Pipeline CI/CD — GitHub Actions

Le pipeline (.github/workflows/ci.yml, 145 lignes) se déclenche sur chaque push et pull request vers develop et master. Il garantit qu'aucun code défaillant ne peut atteindre la production.

Job	Étapes	Bloquant	Dépendance
1 — API Build & Test	Setup .NET 9 → Restore → Build Release → 39 tests xUnit	Oui	Aucune
2 — Web Lint & Build	Setup Node 20 → npm ci --ignore-scripts → ESLint → Vite build	Oui	Aucune
3 — Security Trivy	2 scans CVE CRITICAL/HIGH : NuGet (backend) + npm (frontend)	Non	Job 1
4 — Quality SonarCloud	Analyse statique + Quality Gate (bloquant)	Oui	Job 1
5 — Deploy Gate	Approbation manuelle (GitHub Environments « production »)	Oui	Jobs 1–4

Les jobs 1 et 2 s'exécutent en parallèle (~55 s et ~40 s) ; le pipeline complet jusqu'au gate s'exécute en environ 2 minutes. Le job Security couvre désormais la couche SCA (Software Composition Analysis) sur les deux composants déployés — rôle qu'un outil dédié comme Snyk jouerait dans d'autres architectures, assuré ici sans compte externe supplémentaire. Trivy opère en mode fs (fichiers de dépendances) et non en mode image : aucune image Docker n'est construite par le projet, Railway générant le conteneur automatiquement via Railpack.

2.5 Verrou de déploiement — GitHub Environments

Le déploiement en production est protégé par un verrou à trois niveaux :

1. Conditions techniques — les 4 jobs CI doivent être verts (tests, lint, Trivy, SonarCloud)
2. Condition de branche — le job Deploy ne se déclenche que sur un push vers master (jamais sur une PR)
3. Condition organisationnelle — approbation manuelle requise via GitHub Environments (environnement « production » avec Required reviewer configuré)

Ce mécanisme garantit qu'aucune régression technique et aucune décision unilatérale ne peuvent atteindre la production : même si Railway est techniquement capable de redéploier à chaque push, le gate matérialise un point d'arrêt humain audité.

2.6 Validation du pipeline par la démonstration

Un pipeline qui n'a jamais été vu en train d'échouer n'a jamais prouvé qu'il protège quoi que ce soit. Trois branches de démonstration ont donc été créées volontairement :

PR	Scénario	Résultat observé
#15 — demo/pipeline-ok	Cas nominal : code valide	4 jobs verts, Deploy en attente d'approbation
#16 — demo/pipeline-fail-tests	Test unitaire volontairement incohérent (seuil 100 au lieu de 150)	Job API rouge — jobs suivants ignorés, merge impossible
#17 — demo/pipeline-fail-security	Action Trivy dépinglée (@master) + exit-code bloquant	Trivy rouge + SonarCloud signale le tag mutable — merge impossible

Le scénario #17 illustre la défense en profondeur : deux outils indépendants détectent deux problèmes distincts sur la même modification. Au-delà de ces démonstrations volontaires, l'exploitation réelle a rencontré plusieurs incidents de configuration — build Nixpacks en échec (encodage UTF-8), EBUSY sur un point de montage Railway, changement de syntaxe de serve@14, protection de branches à restaurer, conflit entre analyse automatique et analyse CI SonarCloud — chacun diagnostiqué, corrigé et documenté dans une trace écrite versionnée (docs-bloc3/traces/).

2.7 Déploiement Railway — rollback et continuité

L'application est déployée sur Railway (PaaS) : deux services applicatifs (cesizen-api et cesizen-web) plus un service MySQL managé. Les migrations Entity Framework Core s'appliquent automatiquement au démarrage de l'API — la base de données est toujours synchronisée avec le code déployé.

Le rollback est disponible en un clic depuis le tableau de bord Railway (onglet Deployments → Redeploy sur une version antérieure), avec un temps de reprise inférieur à 2 minutes. RTO : 2 heures (y compris diagnostic et validation). RPO : 24 heures (fréquence des sauvegardes Railway).

Le choix de Railway résulte d'un comparatif avec Azure App Service (MySQL bloqué dans les régions EU testées avec Azure for Students), AWS EC2+RDS (complexité IAM/VPC disproportionnée pour un développeur seul) et VPS+Docker Compose (administration complète du serveur à assumer). Railway offre le meilleur rapport valeur/complexité du contexte : CI/CD natif sur push, HTTPS automatique, base managée, rollback intégré. Pour un contexte d'équipe à plus forte charge, Docker+Kubernetes resterait la cible à moyen terme — évolution documentée, non implémentée par proportionnalité au besoin.

3. Plan de maintenance

3.1 Outil de gestion des tickets — GitHub Issues

GitHub Issues est l'outil de gestion des évolutions et incidents retenu pour CESiZen. Il est configuré avec un système de labels et de milestones pour organiser et prioriser le travail selon un flux structuré prestataire ↔ client — et il est intégré au monitoring automatique (voir 3.3) qui y ouvre et ferme des tickets sans intervention humaine.

Label	Usage	Couleur
bug	Dysfonctionnement identifié	Rouge
enhancement	Nouvelle fonctionnalité ou amélioration	Bleu
security	Vulnérabilité ou risque de sécurité	Orange
documentation	Mise à jour de la documentation	Vert clair
veille	Information issue de la veille technologique	Gris

Label	Usage	Couleur
priority: high	Traitement urgent requis	Rouge foncé
incident / monitoring	Incident ouvert automatiquement par le SIEM	Violet

Milestones : v1.0.0 (livrable Bloc 2 — 19/05/2026), v1.1.0 (déploiement et sécurisation Bloc 3 — juillet 2026), v2.0.0 (évolutions futures : reset de mot de passe par email, rate limiting, export RGPD). Chaque milestone regroupe les Issues et PR associées et offre une vue consolidée de ce qui a été livré à chaque étape.

3.2 Méthodologie de gestion — flux prestataire ↔ client

Dans le contexte de CESIZen, le prestataire est le développeur et le client est l'utilisateur final ou l'organisme commanditaire. Le traitement d'un ticket suit 6 étapes :

#	Étape	Action	Responsable	Critère de sortie
1	Détection	Création du ticket avec description et contexte	Client / Dev	Ticket créé + labellisé
2	Triage	Classification label, milestone, priorité	Développeur	Ticket priorisé
3	Développement	Branche dédiée (fix/ ou feature/), approche TDD	Développeur	Code + tests verts
4	PR + CI	Pull Request, tous les checks CI passent	Développeur	CI verte
5	Merge + Deploy	Merge master, déploiement Railway automatique	Développeur	En production
6	Clôture	Ticket fermé, lié au commit/PR, client notifié	Développeur	Ticket fermé

La traçabilité est systématique : chaque ticket fermé référence le commit ou la Pull Request qui le résout, et chaque action significative du projet donne lieu à une trace écrite horodatée versionnée dans docs-bloc3/traces/ — matière première du présent dossier.

Exemple de cycle réel entièrement automatisé : lorsque le monitoring détecte une indisponibilité de l'API (code non-200), l'issue d'incident est ouverte automatiquement avec la procédure de rollback pré-remplie (étape 1) ; le développeur diagnostique via les logs Railway (étape 2), applique le correctif ou le rollback (étapes 3-5) ; au retour du service, le prochain passage du monitoring ferme l'issue avec un commentaire horodaté (étape 6) — le post-mortem reste attaché au ticket.

3.3 Monitoring automatisé — SIEM

Un workflow GitHub Actions dédié (monitoring.yml, distinct du pipeline CI) surveille la disponibilité de l'API de production en continu. Là où ci.yml vérifie le code avant qu'il n'atteigne la production, monitoring.yml vérifie que l'application déjà déployée répond toujours.

- Fréquence — toutes les 15 minutes, 24h/24 (cron GitHub Actions) + déclenchement manuel possible
- Mécanisme — GET /api/StressEvents en production : l'endpoint traverse toute la chaîne (API + base de données)
- Incident automatique — si code non-200 : création d'une GitHub Issue (labels incident, monitoring, priority: high) contenant l'horodatage, le code HTTP et la procédure de rollback pré-remplie
- Résolution automatique — au retour du 200, le ticket est fermé avec un commentaire horodaté ; un anti-doublon empêche les alertes répétées pour un même incident
- Sécurité — l'action github-script est épinglée sur son hash SHA complet

Indicateurs cibles : MTTD < 15 minutes (fréquence du cron), MTTR < 2 heures (rollback Railway < 2 min + diagnostic), disponibilité > 99,5 %. Ce dispositif constitue une implémentation fonctionnelle de SIEM

proportionnée au contexte ; la roadmap v2 (UptimeRobot externe, tableau de bord Grafana, alertes sur échecs d'authentification) est documentée dans le plan de surveillance dédié.

3.4 Versioning sémantique

Le versioning suit le schéma SemVer (MAJOR.MINOR.PATCH) : MAJOR pour un changement incompatible d'API ou de schéma de données, MINOR pour une fonctionnalité rétrocompatible, PATCH pour une correction sans impact d'interface. Les versions sont gérées via tags Git et GitHub Milestones ; chaque version est identifiable dans Railway, permettant un rollback précis vers n'importe quelle version antérieure.

4. Plan de sécurisation

4.1 Modélisation des menaces — STRIDE

La modélisation des menaces a été réalisée selon la méthode STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), développée par Microsoft. Elle complète l'analyse OWASP en raisonnant par composant et par type d'attaquant.

Composant	Menaces analysées	Couverture	Exemples
API .NET (Controllers)	8 menaces	7/8 — 87%	JWT forgé, injection, bypass RBAC
Base de données MySQL	5 menaces	4/5 — 80%	Injection SQL, accès direct, perte de données
Frontend React	6 menaces	5/6 — 83%	XSS, token volé, CSRF
Pipeline CI/CD	5 menaces	4/5 — 80%	Supply chain, secrets exposés, build compromis
Infrastructure Railway	4 menaces	2/4 — 50%	DoS, accès console, fuite de variables

Total : 28 menaces analysées sur 5 composants, 3 cycles de révision (R1 → R2 → R3). Taux de couverture global : 78,6 %. Les menaces non couvertes sur l'infrastructure sont des risques résiduels acceptés — Railway assume contractuellement la sécurité de la couche infrastructure (modèle de responsabilité partagée du PaaS).

4.2 Analyse des vulnérabilités — OWASP Top 10

L'analyse de sécurité a été réalisée selon le référentiel OWASP Top 10 (2021), le standard de sécurité applicative web recommandé notamment par l'ANSSI. La criticité est calculée selon la formule : Criticité = Probabilité × Impact (échelle 1–3).

Réf	Vulnérabilité	Prob.	Impact	Score	Statut
A01	Broken Access Control	2	3	6	Couvert — JWT RBAC
A02	Cryptographic Failures	1	3	3	Couvert — JWT secret, HTTPS
A03	Injection	1	3	3	Couvert — EF Core ORM
A04	Insecure Design	2	2	4	Partiel — rate limiting planifié
A05	Security Misconfiguration	2	2	4	Partiel — CORS à restreindre
A06	Vulnerable Components	1	3	3	Couvert — Trivy CI x2 (NuGet + npm)
A07	Authentication Failures	2	3	6	Partiel — anti-bruteforce planifié
A08	Data Integrity Failures	1	3	3	Couvert — SHA pinning CI
A09	Logging & Monitoring	2	2	4	Partiel — SIEM en place, logs applicatifs planifiés
A10	SSRF	1	1	1	N/A — aucun appel sortant

Taux de couverture : 90 % (9/10 traités — 5 entièrement couverts, 4 partiels). Le seuil requis de 75 % est largement dépassé.

4.3 Défense en profondeur — chaîne d'analyse

La sécurité est vérifiée à plusieurs moments du cycle de vie, par des outils indépendants — un défaut qui échappe à une couche est rattrapé par la suivante :

Couche	Moment	Outil CESIZen	Statut
SAST — analyse statique du code	Chaque PR	SonarCloud (Quality Gate bloquant)	En place
SCA — dépendances	Chaque CI	Trivy x2 : NuGet (backend) + npm (frontend)	En place
Image de conteneur	Build	Non applicable — aucune image construite (PaaS Railway/Railpack)	Délégué au PaaS
DAST — application en exécution	Post-déploiement	Non outillé (OWASP ZAP envisagé)	Perspective v2
Supervision continue	Toutes les 15 min	SIEM monitoring.yml (tickets automatiques)	En place

4.4 Actions correctives implémentées

- A01 — Contrôle d'accès : JWT RBAC avec rôles USER/ADMIN sur tous les endpoints sensibles ([Authorize(Roles="ADMIN")] sur 10 endpoints d'administration)
- A02 — Cryptographie : clé JWT HMAC-SHA512 (64+ caractères) sortie du code source → GitHub Secrets + Railway Variables ; HTTPS automatique Railway ; mots de passe hachés BCrypt
- A03 — Injection : Entity Framework Core exclusivement (requêtes paramétrées), zéro SQL brut dans la base de code
- A06 — Composants vulnérables : double scan Trivy à chaque CI — dépendances NuGet du backend ET dépendances npm du frontend (couche SCA complète sur les deux composants déployés)
- A08 — Intégrité de la chaîne CI : actions GitHub tierces épinglées sur hash SHA complet, npm ci --ignore-scripts, secrets injectés via blocs env (jamais interpolés dans les commandes)
- A09 — Monitoring : SIEM opérationnel (détection < 15 min, tickets d'incident automatiques — voir 3.3)

4.5 Actions correctives planifiées (v2.0.0)

- A07 — Rate limiting : protection brute-force sur /api/Users/login (middleware ASP.NET Core)
- A07 — Complexité des mots de passe : validation côté API (longueur minimale, caractères spéciaux)
- A05 — Restriction CORS : en production, restreindre à l'URL Railway du frontend (au lieu de AllowAnyOrigin)
- A09 — Journalisation applicative : logs des échecs d'authentification et des actions admin
- A04 — Reset de mot de passe : réinitialisation par email (token temporaire, service SMTP)

4.6 Authentification et contrôle d'accès — fonctionnement

Le mécanisme complet d'authentification illustre les bonnes pratiques de développement appliquées :

4. Inscription : le mot de passe n'est jamais stocké — il est haché avec BCrypt (sel intégré, résistant au brute-force) avant insertion en base
5. Connexion : après vérification BCrypt, l'API génère un JWT signé HMAC-SHA512 contenant trois claims (identifiant, email, rôle) et expirant après 1 heure (ClockSkew zéro — expiration stricte)

6. Chaque requête : le frontend transmet le token (header Authorization: Bearer) ; le middleware ASP.NET Core valide signature, émetteur, audience et expiration avant tout traitement
7. Contrôle d'accès : les attributs [Authorize] / [Authorize(Roles="ADMIN")] sur les contrôleurs appliquent le RBAC côté serveur — masquer un bouton côté client ne suffit jamais, c'est l'API qui refuse (403)

Un utilisateur ne peut ni forger ni modifier un token : sans la clé secrète (absente du code source), toute altération invalide la signature.

Références précises dans le code, consultables en direct lors de la soutenance :

Élément	Emplacement
Génération du token (login)	UserController.cs, méthode Login(), ligne 57
Création du JWT (claims, signature HMAC-SHA512)	UserController.cs, méthode CreateToken(), ligne 172
Validation du token à chaque requête	Program.cs, AddJwtBearer(), ligne 44
RBAC — endpoints réservés ADMIN	UserController.cs (4 endpoints), StressEventsController.cs (3), ResourcesController.cs (4)
RBAC — endpoints réservés utilisateur connecté	UserController.cs : GetMe() ligne 73, ChangePassword() ligne 93

4.7 Protection des données personnelles — RGPD

CESIZen collecte des données personnelles dont certaines sont des données de santé au sens de l'article 9 du RGPD (résultats de diagnostic de stress). Les mesures en place :

Donnée	Catégorie RGPD	Mesure de protection
Email	Donnée personnelle	HTTPS, accès restreint
Mot de passe	Donnée personnelle sensible	Haché BCrypt, jamais stocké en clair
Rôle utilisateur	Donnée de gestion	JWT claims, gestion ADMIN uniquement
Résultats Holmes & Rahe	Donnée de santé (Art. 9)	Accès restreint à l'utilisateur concerné, séparation stricte dev/prod

- Durée de rétention : 2 ans maximum pour les diagnostics ; suppression en cascade compte + diagnostics
- Droits implémentés : suppression du compte et de toutes les données associées (via l'admin)
- Séparation des environnements : aucune donnée de production en développement
- Limites identifiées : absence de droit à la portabilité (export) et d'auto-suppression de compte — planifiés en v2.0.0
- Notification CNIL : obligation Art. 33 RGPD — notification dans les 72 heures en cas de violation de données personnelles

4.8 Gestion de crise

Quatre scénarios de crise sont documentés avec leurs procédures de réponse, leurs niveaux de sévérité et leurs délais de réaction :

Scénario	Déclencheur	Sévérité	Délai	Action principale
Fuite de données	Accès non autorisé BDD	P1 — Critique	< 30 min	Isolation Railway + rotation credentials + CNIL 72h
Compromission CI/CD	Pipeline anormal, secrets exposés	P1 — Critique	< 30 min	Désactivation pipeline + reset secrets + audit commits

Scénario	Déclencheur	Sévérité	Délai	Action principale
Indisponibilité service	Erreurs 5xx persistantes	P2 — Majeur	< 2 h	Vérif Railway status → Rollback Redeploy
Tentative d'intrusion	Alertes Trivy / SonarCloud / logs	P3 — Mineur	< 24 h	Analyse logs → Ticket security → Hotfix

L'escalade suit cinq phases : détection (souvent automatique via le SIEM), confinement, correction, communication (notification CNIL sous 72 h si des données personnelles sont concernées — Art. 33 RGPD), retour d'expérience. RTO : 2 heures | RPO : 24 heures. Chaque incident donne lieu à un post-mortem documenté dans GitHub Issues (chronologie + cause racine).

Les responsabilités et canaux de communication sont définis à l'avance pour éviter toute improvisation en situation de crise :

Acteur	Responsabilité	Canal
Développeur (prestataire)	Détection, diagnostic, confinement, correction, post-mortem	GitHub Issues + logs Railway
Commanditaire (client)	Informé sous 2 h pour tout incident P1/P2 ; valide la reprise	Email + point de suivi
CNIL	Notifiée sous 72 h en cas de violation de données personnelles	Téléservice de notification CNIL
Utilisateurs concernés	Informés sans délai indu si le risque pour leurs droits est élevé (Art. 34)	Email
Support Railway / GitHub	Sollicité si l'incident relève de la plateforme	Tickets support officiels

4.9 Sécurité de la chaîne d'approvisionnement (supply chain)

Une attaque supply chain ne cible pas le code de CESIZen directement : elle compromet un maillon dont le projet dépend (une librairie, une action CI, un paquet) pour faire transiter du code malveillant sans qu'il ait été écrit par le développeur. Des cas réels documentés (SolarWinds 2020, backdoor xz-utils 2024, packages npm piégés) illustrent l'ampleur du risque. Trois défenses concrètes sont en place :

Risque	Où il peut survenir	Défense mise en place
Action GitHub tierce réécrite après coup	Un tag mutable (@master) peut être redirigé vers du code malveillant	SHA pinning — ci.yml ligne 78 : épinglage sur le hash de commit complet, jamais un tag
Script d'installation npm malveillant	postinstall peut exécuter du code arbitraire à l'installation	npm ci --ignore-scripts — ci.yml ligne 56
Dépendance avec CVE connue	Un paquet NuGet ou npm intégré au projet contient une faille publiée	Trivy — scan CRITICAL/HIGH sur NuGet (backend) et npm (frontend) à chaque CI

La démonstration n'est pas que théorique : la PR de démonstration demo/pipeline-fail-security (#17) remplace volontairement le SHA pinning par @master, et le pipeline le détecte — Trivy signale la CVE introduite, SonarCloud signale indépendamment le tag mutable comme Security Hotspot. Deux outils distincts détectent deux problèmes distincts sur la même modification, preuve concrète de la défense en profondeur.

4.10 Bonnes pratiques de développement

- TDD systématique — le test précède le code (Red → Green → Refactor) ; 47 tests automatisés couvrant cas nominaux, bornes et cas d'échec

- Revue par Pull Request — aucun code n'atteint develop ou master sans passer les 4 vérifications CI
- Conventions — commits préfixés (feat:, fix:, security:...), branches nommées par type, code et commentaires en anglais
- Traçabilité — chaque action significative (config, déploiement, choix technique, incident) documentée dans une trace horodatée versionnée
- Séparation configuration/code — aucun secret ni paramètre d'environnement codé en dur

5. Veille technologique

5.1 Méthode et organisation

La veille technologique est structurée sur trois fréquences complémentaires, combinant surveillance manuelle et automatisation CI :

Fréquence	Type	Outils / Sources
Quotidienne	Alertes CVE, flux RSS sécurité	NVD, ANSSI, GitHub Security Advisories
Hebdomadaire	Newsletters, articles techniques	TLDR Security, JavaScript Weekly, GitHub Changelog
Mensuelle	Release notes, guides de sécurité	devblogs.microsoft.com, react.dev, owasp.org
Continue (CI)	Analyse automatisée à chaque commit	Trivy x2 (CVE), SonarCloud (qualité), Dependabot

L'outillage CI constitue la pierre angulaire de la veille : Trivy détecte automatiquement les nouvelles CVE sur les dépendances backend et frontend à chaque build, SonarCloud signale les régressions de qualité et de sécurité, Dependabot propose des mises à jour de dépendances directement dans les Pull Requests.

5.2 Sources principales

Domaine	Source	Fréquence
Sécurité CVE	NVD (NIST), GitHub Security Advisories, ANSSI	Quotidien
OWASP	owasp.org — guides, cheat sheets, Top 10	Mensuel
.NET / ASP.NET Core	devblogs.microsoft.com/dotnet	Mensuel
React / Frontend	react.dev/blog, JavaScript Weekly	Hebdomadaire
CI/CD & DevSecOps	GitHub Changelog, TLDR Security	Hebdomadaire
JWT / Authentification	auth0.com/blog, jwt.io, RFC IETF	Mensuel

5.3 Actions concrètes issues de la veille

Chaque action de sécurité implémentée dans CESiZen est directement traçable à une information de veille :

- Supply chain attacks (2024) — épinglage SHA complet de toutes les actions GitHub tierces (aquasecurity/trivy-action, actions/github-script...)
- npm script injection — ajout de --ignore-scripts à npm ci dans le pipeline, et extension du scan de CVE aux dépendances npm du frontend
- Secrets dans les logs CI — injection par variables d'environnement (\$SONAR_TOKEN, \$JWT_KEY) au lieu d'interpolation directe dans les commandes
- OWASP Top 10 2021 — analyse complète appliquée à CESiZen avec plan d'actions correctives documenté
- RGPD Art. 9 — identification et protection renforcée des résultats de diagnostic comme données de santé
- HMAC-SHA512 key length — clé JWT portée à 64+ caractères (512 bits minimum requis par la RFC 7518)

- Dépréciation de Create React App (2023) — adoption de Vite comme outil de build moderne du frontend

6. Conclusion et couverture des critères

Ce dossier technique présente la mise en œuvre du Bloc 3 « Déployer et sécuriser les applications informatiques » appliquée au projet CESiZen. L'ensemble des critères de la grille d'évaluation est couvert :

Critère Bloc 3	Statut	Éléments de preuve
1 environnement déployé avec automatisations	Atteint	Railway (API + Web + MySQL) + CI/CD GitHub Actions 5 jobs
Versioning configuré	Atteint	GitHub + Git Flow + protection de branches (enforce_admins, 4 checks)
Outil de ticketing configuré	Atteint	GitHub Issues (7 labels, 3 milestones, tickets SIEM automatiques)
Analyse vulnérabilités + criticité	Atteint	OWASP Top 10 (90 %) + STRIDE (28 menaces, 78,6 %)
Actions correctives ≥ 75 % des risques	Atteint	6 implémentées, 5 planifiées — 90 % de couverture
Gestion de crise documentée	Atteint	4 scénarios P1/P2/P3 avec procédures et escalade en 5 phases
Données personnelles et RGPD	Atteint	Registre, Art. 9, droits, rétention, notification 72 h
Veille technologique	Atteint	Sources multiples + CI automatisé + 7 actions concrètes appliquées

La démonstration en soutenance s'appuiera sur l'environnement Railway déployé, le pipeline GitHub Actions opérationnel (dont 3 scénarios de démonstration : pipeline vert, échec sur test, échec sur sécurité), et le monitoring SIEM actif.

Les limites du projet sont identifiées et assumées plutôt que masquées : tests frontend hors pipeline CI, absence de scan d'image de conteneur (aucune image construite — couche déléguée au PaaS) et de DAST, CORS à restreindre, reset de mot de passe par email non implémenté. Chacune dispose d'une évolution documentée dans les milestones GitHub et constitue le backlog de la version v2.0.0 — démontrer que l'on sait ce qui reste à faire, pourquoi, et comment le faire, fait partie intégrante d'une démarche professionnelle de sécurisation.

Annexe A — Documents complémentaires

Ce dossier constitue la synthèse autoporteuse des trois plans exigés. Chaque volet dispose d'une version détaillée en document séparé, disponible dans docs-bloc3/ :

Document	Contenu
Plan_Deploiement_CESIZen.docx	Détail complet du plan de déploiement : environnements, pipeline, incidents réels, comparatif d'hébergement
Analyse_OWASP_CESIZen.docx	Détail de l'analyse des 10 vulnérabilités OWASP avec calcul de criticité
Modelisation_Menaces_STRIDE_CESIZen.docx	Détail des 28 menaces STRIDE sur les 5 composants
Plan_RGPD_CESIZen.docx	Registre des traitements, droits des personnes, mesures de protection
Plan_Gestion_Crise_CESIZen.docx	Procédures détaillées des 4 scénarios de crise
Plan_Surveillance_SIEM_CESIZen.docx	Architecture du monitoring, métriques, roadmap v2
Veille_Technologique_CESIZen.docx	Détail des sources et de la méthodologie de veille

Annexe B — Glossaire

Terme	Définition
RBAC	Role-Based Access Control — contrôle d'accès basé sur les rôles (USER / ADMIN dans CESIZen)
JWT	JSON Web Token — jeton d'authentification signé, transmis à chaque requête (header Authorization: Bearer)
SHA (cryptographie)	Secure Hash Algorithm — famille de fonctions de hachage ; ici HMAC-SHA512 signe les JWT
SHA pinning	Référencer une dépendance (action GitHub) par son empreinte de commit complète plutôt qu'un tag mutable
SAST	Static Application Security Testing — analyse statique du code source (SonarCloud)
SCA	Software Composition Analysis — analyse des dépendances tierces (Trivy, rôle habituellement tenu par Snyk)
DAST	Dynamic Application Security Testing — test de l'application réellement déployée (non outillé, perspective v2)
PaaS	Platform as a Service — plateforme gérant l'infrastructure sous-jacente (Railway)
SIEM	Security Information and Event Management — supervision et corrélation d'événements de sécurité
MTTD / MTTR	Mean Time To Detect / Repair — délai moyen de détection / résolution d'un incident
RTO / RPO	Recovery Time/Point Objective — délai de reprise et perte de données maximale tolérée
CVE	Common Vulnerabilities and Exposures — identifiant standard d'une vulnérabilité publiée
Quality Gate	Ensemble de règles SonarCloud qu'une analyse doit respecter pour être considérée réussie